SOFTWARE ENGINEERING

LAB 03

NAME : BHAKTI S PATIL

SRN : PES2UG24CS112

SECTION : 5B CSE

LAB 3 : ARCHITECTURE SELECTION & JUSTIFICATION

Remote Team Time-Tracking & Project Approver

Architecture Selection :

I chose Layered Architecture for the Remote Team Time-Tracking & Project Approver System. The system is organized into three horizontal layers: User-Facing Layer, Business Logic Layer, and Data Layer. This matches the system's separation between user interaction, business processing, and persistent project/timesheet data.

Requirements and Main Challenges :

The main functional requirements are recording developer hours against project tasks, preventing more than 24 hours in a calendar day, submitting timesheets for approval, allowing the Engineering Manager to approve or reject them, providing project budget burn-rate information, and restricting information according to user roles. The main architectural challenges are maintaining security through role-based access and providing timely dashboard updates; the stated performance requirement is that approval-status transitions and budget burn charts update within 500 ms under normal operating conditions.

Architectural Style Analysis :

Layered Architecture provides clear separation of concerns and is easy to understand for a traditional application. Microservices would provide independent scaling and fault isolation, but would introduce operational complexity, network latency and data-consistency challenges. Client-Server would provide centralized control and easy data consistency, but introduces a centralized server dependency, scalability bottleneck and limited fault tolerance. For this system, the clear separation and centralized business-rule processing of Layered Architecture are the best fit.

Two Scenario-Specific Reasons :

1. Separation of business rules: Timesheet validation, the 24-hour limit, submission workflow, approval/rejection and role-based access can be kept in the Business Logic Layer, while the user interface and database remain separate. This makes the approval workflow easier to maintain and change.
2. Clear data and service organization: Project/timesheet queries and recorded hours are handled through the Data Layer, while budget burn-rate and approval services remain in the Business Logic Layer. This provides a clear path between user requests, business processing and stored data.

Component Identification :

The five components are Remote Developer Interface, Order Manager Component, Timesheet Validation & Approval Component, Payment Service Component, and Project & Timesheet Database. The Order Manager and Payment Service are the two components specified by the handout; the remaining three components complete the architecture for user interaction, validation/approval and persistent data storage.

Interface and Dependency Design :

The component diagram shows at least four interfaces using provided/required UML notation and labels the service interactions: Timesheet Validation / Approval API, Timesheet & Approval Data Access, Budget Burn-Rate API, and Recorded Hours & Budget Data. The diagram also shows the relevant component dependencies and data flow between the business components and database.

Security Advantage :

Layered Architecture allows role-based access control to be enforced in the Business Logic Layer before protected timesheet and project information is accessed. Developers can be limited to their own records, while Engineering Managers can access information required for approval and project monitoring. Centralizing authorization helps prevent unauthorized viewing or modification of project and timesheet data.

Performance Benefit :

Although Layered Architecture can introduce some processing overhead between layers, this design keeps responsibilities organized and avoids duplicating validation, approval and budget calculations. Centralized business processing can be optimized independently, while the Data Layer can be accessed through focused queries. This supports the requirement for dashboard approval-status and budget updates within 500 ms under normal operating conditions.

UML COMPONENT DIAGRAM

